

Instalación del framework V2X

VaN3Twin (ns-3) + SUMO + visor 3D SUMO-GEO, con Docker, en Linux, Windows y macOS

Universidad de Cuenca — Redes Vehiculares y Heterogéneas (INGE-00104)

Prof. Pablo Barbecho — 28 de agosto de 2026

Contenido

1	Objetivo	1
2	Marco teórico	2
2.1	Los componentes del framework	2
2.2	Los repositorios y qué papel juega cada uno	2
2.3	¿Por qué Docker? Imagen, contenedor y volumen	3
2.4	<code>docker</code> vs. <code>docker compose</code> , y desde qué carpeta ejecutarlos	3
2.5	Los ficheros del proyecto: qué es cada uno y cuándo lo usamos	4
2.6	El problema de las mayúsculas (léelo antes de clonar)	4
2.7	Repaso exprés de git	4
2.8	Repaso exprés de Docker y Compose	5
3	Trabajo previo (obligatorio, antes de la clase)	5
4	Instalación paso a paso por sistema operativo	5
4.1	Linux (Ubuntu/Debian)	5
4.2	Windows 10/11 (con WSL2)	6
4.3	macOS (Intel o Apple Silicon)	6
4.4	Pasos comunes: clonar y construir	7
5	Verificación por componentes (cada pieza por separado)	7
5.1	Nivel 0 — el contenedor	7
5.2	Nivel 1 — SUMO solo (movilidad, sin red)	7
5.3	Nivel 2 — VaN3Twin/ns-3 solo (red + movilidad, sin visor)	8
5.4	Nivel 3 — SUMO-GEO solo (visor autónomo, sin VaN3Twin)	8
5.5	Nivel 4 — Integración completa (todo junto)	8
5.6	Nivel 5 — Replay de mensajes (opcional, cierra el círculo)	9
6	Problemas frecuentes	10
7	Entregable	10

1 Objetivo

Instalar y poner en marcha, en tu propio computador, el entorno completo de simulación V2X del curso: el framework **VaN3Twin** (comunicaciones vehiculares sobre ns-3), el simulador de movili-

dad **SUMO** y el visor web 3D **SUMO-GEO**. Al terminar sabrás verificar cada componente por separado y ejecutar una simulación integrada viéndola en vivo en el navegador — la base sobre la que trabajan las demás prácticas del curso.

2 Marco teórico

2.1 Los componentes del framework

- **SUMO** (*Simulation of Urban MObility*): simulador microscópico de tráfico. Aporta la *movilidad*: mapas, rutas y el movimiento de cada vehículo.
- **ns-3 + VaN3Twin**: ns-3 es el simulador de redes de referencia en investigación; VaN3Twin (evolución de ms-van3t, Politecnico di Torino) le añade la pila vehicular ETSI ITS-G5 completa (mensajes CAM/DENM/CPM reales en ASN.1) sobre 802.11p, LTE, C-V2X o NR-V2X.
- **TraCI** (*Traffic Control Interface*): el puente. ns-3 se conecta a SUMO por un socket TCP y ambos avanzan sincronizados: SUMO mueve los vehículos, ns-3 simula sus comunicaciones.
- **SUMO-GEO**: visor web (backend Python FastAPI + frontend MapLibre/deck.gl) que se conecta a SUMO como *segundo* cliente TraCI de solo lectura y dibuja la simulación en 3D sobre el mapa real, en el navegador. Incluye además un modo *replay* que reproduce corridas grabadas (.pcap) con el intercambio de mensajes.

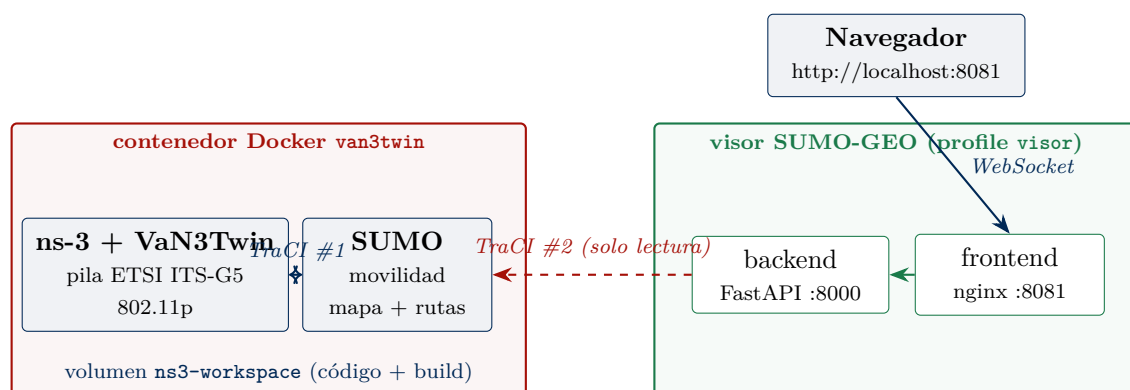


Figure 1: Arquitectura en ejecución: SUMO acepta dos clientes TraCI (ns-3 controla; el visor solo lee) y avanza en *lockstep* con ambos.

2.2 Los repositorios y qué papel juega cada uno

Todo se despliega desde GitHub (figura 2). Solo se clonan **dos** repos — el tercero (el código ns-3) lo clona Docker *dentro de la imagen* durante la construcción, lo que además evita un problema serio de sistemas de ficheros (§2.6).

Repo (github.com/Pbarbecho)	Contenido	¿Se clona a mano?
van3twin-docker	Dockerfile, docker-compose.yml, manuales y prácticas	Sí — punto de entrada
SUMO_GEO	Visor web (backend + frontend)	Sí — junto al anterior
VaN3TwinGEO	Fork de VaN3Twin con los parches de la integración	No — lo clona el build de Docker dentro de la imagen

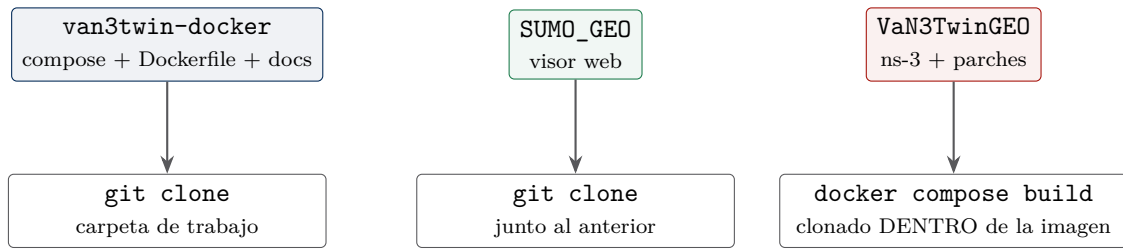


Figure 2: De GitHub a tu máquina: dos clones manuales; el código ns-3 viaja dentro de la imagen Docker.

2.3 ¿Por qué Docker? Imagen, contenedor y volumen

El entorno nativo de VaN3Twin exige Ubuntu, decenas de dependencias y horas de compilación. Docker lo empaqueta: la **imagen** es una plantilla inmutable (Ubuntu 22.04 + SUMO + gRPC + ns-3 *ya compilado*); un **contenedor** es una instancia en ejecución de esa imagen; un **volumen** es un disco persistente gestionado por Docker — aquí, el volumen `ns3-workspace` guarda el árbol de código y sobrevive aunque el contenedor se destruya. **Docker Compose** describe en un YAML los servicios (contenedores), sus redes, puertos y volúmenes, y los orquesta con un solo comando. Los *profiles* permiten grupos opcionales: aquí el visor solo arranca si se pide `--profile visor`.

2.4 docker vs. docker compose, y desde qué carpeta ejecutarlos

Son dos niveles de la misma herramienta:

- **docker a secas** opera sobre objetos *globales* del demonio (imágenes, contenedores, volúmenes, identificados por nombre). Sirve para acciones puntuales: `docker images`, `docker ps`, `docker exec -it van3twin bash`, `docker logs`. Como los nombres son globales, **funciona desde cualquier carpeta**.
- **docker compose** opera sobre un *proyecto*: el conjunto de servicios, redes y volúmenes descrito en un `docker-compose.yml`. Por eso **debe ejecutarse en la carpeta donde está ese fichero** (o señalarlo con `-f ruta/al/compose.yml`). Si lo lanzas en otra carpeta, obtendrás `configuration file provided: not found`; y si en esa otra carpeta hubiera *otro* compose, estarías manejando **otro proyecto distinto** sin darte cuenta.

En este curso eso significa: los `docker compose ...` del stack se lanzan desde `van3twin-docker/`; el del visor autónomo (nivel 3 de la verificación), desde `SUMO_GEO/`. Son dos proyectos independientes — con contenedores, redes y volúmenes propios — que casualmente comparten los puertos 8000/8081, y por eso no pueden estar arriba a la vez. Algo parecido pasa con `docker build`: la ruta que le pasas (el *contexto*, p.ej. el “.” de `docker build -t x .`) define dónde busca el `Dockerfile` y qué ficheros puede copiar dentro de la imagen; `docker compose build` hace esto solo, leyendo la ruta del YAML.

2.5 Los ficheros del proyecto: qué es cada uno y cuándo lo usamos

Fichero	Qué es	Cuándo lo usamos
van3twin-docker/Dockerfile	Receta de la imagen van3twin : Ubuntu + SUMO + gRPC + clona el fork y compila ns-3, capa a capa	Solo indirectamente, vía docker compose build (trabajo previo); no se edita
van3twin-docker/docker-compose.yml	El fichero central : define los servicios van3twin , backend y frontend (profile visor), el volumen ns3-workspace , puertos y montajes	En cada up / exec / logs / stop del día a día
van3twin-docker/.env	Variables locales de TU máquina (p.ej. SUMO_GEO_DIR); compose lo lee solo	Solo si SUMO_GEO no está junto a van3twin-docker
SUMO_GEO/docker-compose.yml	Proyecto <i>independiente</i> : el visor autónomo (modo <i>managed</i> , escenarios de Cuenca)	Solo en el nivel 3 de verificación; después, down
SUMO_GEO/backend/Dockerfile.van3twin	Receta de la imagen del backend en modo integrado (Python slim + traci 1.12, sin binario SUMO)	Indirectamente: la construye el --profile visor del compose central
SUMO_GEO/backend/Dockerfile	Receta del backend autónomo (incluye SUMO completo)	Indirectamente, solo en el nivel 3
SUMO_GEO/frontend/ (html/js/css + nginx.conf)	El visor que ves en el navegador; nginx lo sirve <i>montado</i> , sin construir imagen	Siempre que abres localhost:8081 ; los cambios se ven con recargar

2.6 El problema de las mayúsculas (léelo antes de clonar)

El repo de ns-3/VaN3Twin contiene pares de ficheros que **solo difieren en mayúsculas** (**ActionID.c**/**ActionId**, **BOOLEAN.h**/**boolean.h**...). En Linux (ext4) son dos ficheros distintos; en los sistemas de ficheros por defecto de **Windows (NTFS)** y **macOS (APFS)**, que no distinguen mayúsculas, uno *machaca* al otro al clonar y el árbol queda corrupto — git incluso avisa (“paths have collided”). Por eso el diseño del framework: **el código ns-3 nunca toca tu disco**; vive dentro de la imagen y del volumen Docker (Linux, case-sensitive). Los dos repos que sí clonas no tienen este problema.

2.7 Repaso exprés de git

Comando	Qué hace
<code>git clone <url></code>	Descarga un repositorio completo (código + historial)
<code>git status</code>	Estado: rama actual, ficheros modificados
<code>git log --oneline -5</code>	Últimos 5 commits
<code>git pull</code>	Trae los cambios nuevos del remoto (actualizarse)
<code>git branch / git checkout <rama></code>	Listar / cambiar de rama
<code>git diff</code>	Ver qué cambió respecto al último commit
<code>git add <f> + git commit -m "..."</code>	Registrar cambios propios
<code>git remote -v</code>	Ver a qué URL apunta el repo

2.8 Repaso exprés de Docker y Compose

Comando	Qué hace
<code>docker images / docker ps -a</code>	Imágenes locales / contenedores (todos)
<code>docker compose build</code>	Construye las imágenes del compose
<code>docker compose up -d</code>	Crea y arranca los servicios (en segundo plano)
<code>docker compose --profile visor up -d</code>	Ídem incluyendo el grupo opcional <code>visor</code>
<code>docker compose ps</code>	Estado de los servicios del proyecto
<code>docker compose exec van3twin bash</code>	Shell dentro del contenedor
<code>docker compose logs <servicio></code>	Logs de un servicio (depuración)
<code>docker compose stop / down</code>	Parar / parar y eliminar contenedores
<code>docker compose down -v</code>	¡Peligro! borra también los volúmenes (el build)
<code>docker volume ls / docker system df</code>	Volúmenes / espacio usado

3 Trabajo previo (obligatorio, antes de la clase)

TRABAJO PREVIO — traer HECHO a la clase

La construcción de la imagen compila ns-3 completo: **1–3 horas** y **~3–4 GB** de descargas. **Es inviable hacerlo en clase.** Trae hecho, según tu sistema operativo (secciones 4.1–4.3):

1. Docker instalado y funcionando (`docker run hello-world`).
2. Los dos repos clonados, **uno junto al otro**.
3. La imagen construida: `docker compose build` terminado sin errores, y `docker images` mostrando `van3twin` de ~10–15 GB.
4. (Ideal) La verificación §5.3 pasada: la simulación de prueba corre.

Recursos mínimos: 4 CPUs, 8 GB de RAM para Docker (recomendado 12–16), 60 GB de disco libre.

4 Instalación paso a paso por sistema operativo

Los pasos de clonado y construcción (§4.4) son idénticos en los tres sistemas; lo que cambia es cómo instalar Docker y *dónde* clonar.

4.1 Linux (Ubuntu/Debian)

TERMINAL

```
# 1) Docker Engine + Compose (repositorio oficial de Docker)
sudo apt-get update && sudo apt-get install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo $VERSION_CODENAME)
  stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update && sudo apt-get install -y docker-ce docker-ce-cli \
  containerd.io docker-compose-plugin

# 2) Usar docker sin sudo (cerrar sesion y volver a entrar despues)
```

```
sudo usermod -aG docker $USER

# 3) Probar
docker run hello-world
```

En Linux puedes clonar en cualquier carpeta (ext4 distingue mayúsculas). Continúa en §4.4.

4.2 Windows 10/11 (con WSL2)

TERMINAL — PowerShell como administrador

```
# 1) Habilitar WSL2 con Ubuntu (reinicia cuando lo pida)
wsl --install
```

2) Instalar **Docker Desktop para Windows** desde <https://www.docker.com/products/docker-desktop/> y, en *Settings* → *General*, verificar que usa el *WSL 2 based engine*; en *Settings* → *Resources* → *WSL integration*, activar la integración con Ubuntu. Asignar recursos en *Resources* (CPUs/RAM según el trabajo previo).

Advertencia

Todo el trabajo se hace **dentro de la terminal de Ubuntu/WSL2** (aplicación “Ubuntu” del menú inicio), y los repos se clonan en el *home* de WSL (~/), **nunca** en /mnt/c/... (eso es el disco NTFS de Windows: E/S lenta y el problema de mayúsculas de §2.6).

TERMINAL — Ubuntu (WSL2)

```
docker run hello-world      # comprueba que Docker Desktop integra con WSL
```

Continúa en §4.4 (desde la terminal de Ubuntu/WSL2).

4.3 macOS (Intel o Apple Silicon)

TERMINAL

```
# 1) Docker Desktop (o descargar el .dmg de docker.com)
brew install --cask docker
open -a Docker      # arrancarlo una vez y aceptar permisos
docker run hello-world
```

En *Docker Desktop* → *Settings* → *Resources*: CPUs y RAM según el trabajo previo. Los dos repos a clonar no tienen problema de mayúsculas, así que cualquier carpeta normal sirve (p.ej. ~/van3twin). Recuerda la advertencia general: el fork de ns-3 **no** se clona en el Mac — lo maneja Docker (§2.6). Continúa en §4.4.

4.4 Pasos comunes: clonar y construir

TERMINAL — igual en los tres sistemas

```
# 1) Carpeta de trabajo con los DOS repos, uno junto al otro
mkdir -p ~/v2x && cd ~/v2x
git clone https://github.com/Pbarbecho/van3twin-docker.git
git clone https://github.com/Pbarbecho/SUMO_GEO.git
ls                # debe mostrar: van3twin-docker SUMO_GEO

# 2) Construir la imagen (1-3 h: hazlo ANTES de clase)
cd van3twin-docker
docker compose build 2>&1 | tee build.log

# 3) Arrancar el contenedor (primer arranque: puebla el volumen, ~1-2 min)
docker compose up -d
docker compose ps      # van3twin -> running
```

Nota

El compose asume que SUMO_GEO está *junto* a van3twin-docker (ruta ../SUMO_GEO). Si lo clonaste en otro sitio, crea un fichero `.env` dentro de `van3twin-docker` con la línea `SUMO_GEO_DIR=/ruta/a/SUMO_GEO`. Si la construcción se interrumpe (red, suspensión), relánzala: continúa desde la última capa cacheada.

5 Verificación por componentes (cada pieza por separado)

La estrategia de depuración del curso: verificar de dentro hacia fuera. Si un nivel falla, no tiene sentido probar el siguiente.

5.1 Nivel 0 — el contenedor

DENTRO DEL CONTENEDOR van3twin

```
docker compose exec van3twin bash      # (desde van3twin-docker/)
whoami                                # vanet
lsb_release -ds                       # Ubuntu 22.04.x LTS
pwd                                   # /home/vanet/VaN3Twin/ns-3-dev
```

5.2 Nivel 1 — SUMO solo (movilidad, sin red)

DENTRO DEL CONTENEDOR van3twin

```
sumo --version                       # Eclipse SUMO Version 1.12.0
# escenario del EVA con SOLO SUMO, 30 s simulados, sin ns-3:
sumo -c src/automotive/examples/sumo_files_v2v_map/map.sumo.cfg \
    --no-step-log -e 30
```

✓ Verificación

Termina en segundos con un resumen tipo `Simulation ended at time: 30.00` y estadísticas de vehículos (*Inserted, Running...*). SUMO funciona.

5.3 Nivel 2 — VaN3Twin/ns-3 solo (red + movilidad, sin visor)

DENTRO DEL CONTENEDOR `van3twin`

```
./ns3 show profile      # build profile: optimized
./ns3 run "v2v-emergencyVehicleAlert-80211p --sumo-gui=false --sim-time=20 --met-sup=true"
```

✓ Verificación

Arranca SUMO internamente (`Starting server on port 34XX`), se ven líneas de recepción de mensajes (`veh2 received a new CPMv2 from 1...`) y al terminar imprime el resumen del *Metric Supervisor* (CAMs, PRR, latencias). La pareja ns-3 ↔ SUMO funciona.

5.4 Nivel 3 — SUMO-GEO solo (visor autónomo, sin VaN3Twin)

El visor tiene un modo autónomo (*managed*) con sus propios escenarios de Cuenca — perfecto para probarlo aislado:

TERMINAL

```
cd ../SUMO_GEO
docker compose up -d --build      # ~3 min la primera vez
```

EN EL NAVEGADOR

<http://localhost:8081> → mapa de **Cuenca** con tráfico simulado, calles coloreadas por congestión y panel de históricos. El visor funciona.

Advertencia

Al terminar esta prueba, **apágalo**: `docker compose down` (desde `SUMO_GEO/`). Usa los mismos puertos 8000/8081 que el modo integrado y no pueden convivir.

5.5 Nivel 4 — Integración completa (todo junto)

TERMINAL

```
cd ../van3twin-docker
docker compose --profile visor up -d      # van3twin + backend + frontend
```


DENTRO DEL CONTENEDOR van3twin

```

pkill -f ns3-dev-v2v; pkill sumo; sleep 1
./ns3 run "v2v-emergencyVehicleAlert-80211p --sumo-gui=false --met-sup=true --sumo-updates
=0.1 --num-traci-clients=2"
# -> "waiting for 2 clients... client connected" y SE QUEDA ESPERANDO (correcto)

```

EN EL NAVEGADOR

<http://localhost:8081> → al conectar, en la terminal aparece el segundo **client connected** y la simulación arranca: mapa de **Turín** con los vehículos moviéndose en 3D (figura 3).

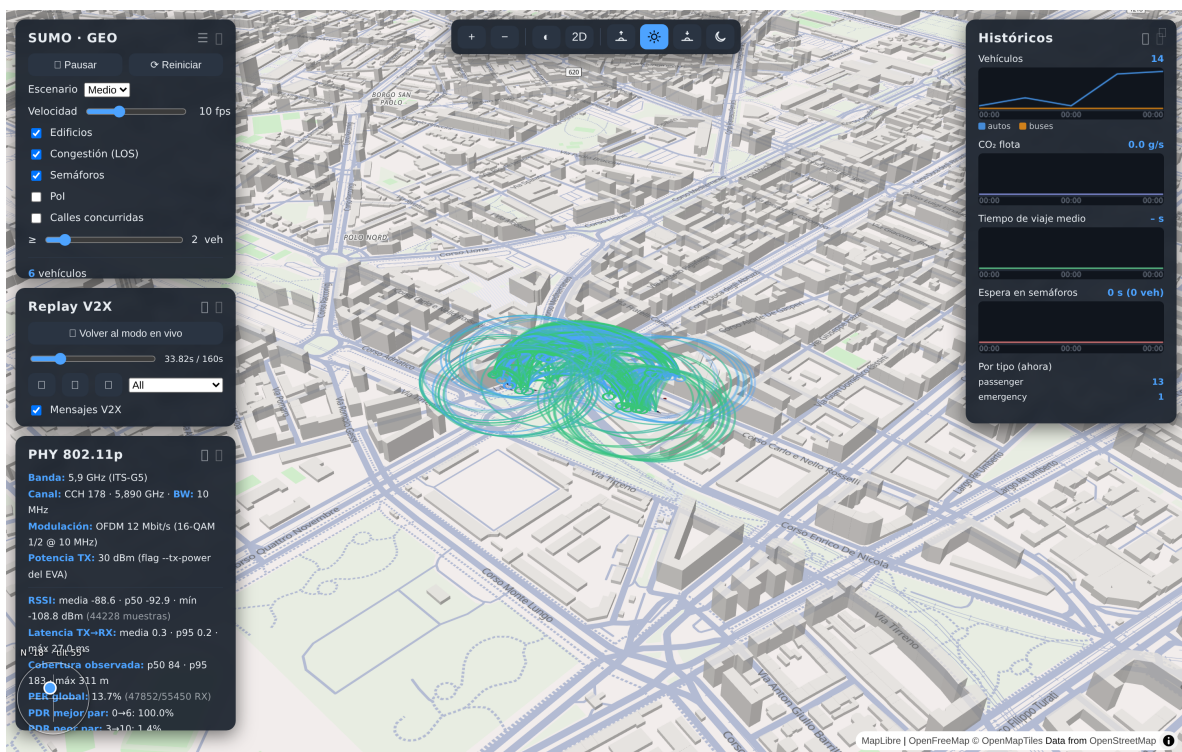


Figure 3: Resultado esperado del nivel 4/5: el visor con la flota del EVA, los pulsos de transmisión y los paneles de control y métricas.

5.6 Nivel 5 — Replay de mensajes (opcional, cierra el círculo)**DENTRO DEL CONTENEDOR van3twin**

```

# tras terminar una corrida del nivel 4:
cp ~/VaN3Twin/ns-3-dev/v2v-EVA-*.pcap ~/results/

```

EN EL NAVEGADOR

Botón **Replay pcap** (panel Replay V2X) → línea de tiempo, paso a paso ◀ / ▶ y clic sobre pulsos/arcs para ver el contenido ASN.1 de cada mensaje. El detalle de este modo es el tema de la práctica *Practica_Replay_V2X.pdf*.

6 Problemas frecuentes

Síntoma	Causa y solución
<code>g++: internal compiler error: Killed</code> durante el build	Falta RAM en Docker: sube a 12–16 GB (Desktop → Resources; en WSL2, fichero <code>.wslconfig</code>) y relanza el build
<code>paths have collided</code> al clonar	Clonaste el fork de ns-3 en NTFS/APFS: no lo clones — lo gestiona Docker (§2.6)
El build va lentísimo en Windows	Repos clonados en <code>/mnt/c</code> : muévelos al home de WSL2 (<code>~/</code>)
<code>port is already allocated</code> al up	El visor autónomo (nivel 3) u otra app usa 8000/8081: <code>docker compose down</code> en SUMO_GEO/
<code>waiting for 2 clients</code> eterno	Falta abrir el navegador (nivel 4); o el visor no está arriba: <code>docker compose --profile visor up -d</code>
La simulación arranca sin esperar al visor	Faltó <code>--num-traci-clients=2</code> en el comando
Visor en negro	<code>docker compose logs backend</code> (desde <code>van3twin-docker/</code>) y <code>curl localhost:8000/api/health</code>
<code>context not found ../SUMO_GEO</code>	Los repos no están uno junto al otro: muévelos o crea el <code>.env</code> con <code>SUMO_GEO_DIR</code>

7 Entregable

✓ Verificación final — capturas a entregar

1. `docker images` mostrando la imagen `van3twin`.
2. Salida del nivel 1 (SUMO) y del nivel 2 (resumen del Metric Supervisor).
3. Navegador en el nivel 3 (Cuenca) y en el nivel 4 (Turín con vehículos).
4. Un mensaje abierto en el nivel 5 (popup con el ASN.1), si llegaste.

Documentos relacionados: `Manual_Simulacion_Van3Twin_SUMO_GEO.pdf` (operación diaria del stack), `Practica_Replay_V` (análisis del intercambio de mensajes). Repos: <https://github.com/Pbarbecho/van3twin-docker>, https://github.com/Pbarbecho/SUMO_GEO, <https://github.com/Pbarbecho/Van3TwinGEO>.